Crio.Do

LEARNING ROADMAP

Java Spring Boot Backend Roadmap

From Java Basics to Production-Ready Microservices

Intensive	Balanced	Flexible
40-50 hrs/week	15-20 hrs/week	8-10 hrs/week

- 7 Phases
- ★ Beginner to Advanced

https://crio.do

Overview

This roadmap takes you from Java fundamentals to building production-grade backend systems with Spring Boot. You'll master object-oriented programming, understand the Spring ecosystem deeply, build RESTful APIs, work with databases, implement security, and learn to deploy scalable microservices. By the end, you'll be equipped to architect and build enterprise-level backend applications.

Who is this for?

Aspiring backend developers, frontend developers wanting full-stack skills, computer science students, developers from other languages transitioning to Java, or anyone wanting to build robust server-side applications.

Prerequisites

- Basic programming concepts (variables, loops, functions)
- Understanding of how the web works (HTTP, client-server)
- Familiarity with any programming language (helpful but not required)
- A computer with JDK installed
- Willingness to read documentation

What you'll learn

- Write clean, object-oriented Java code
- Understand the Spring framework's core concepts (IoC, DI)
- Build RESTful APIs with Spring Boot
- Work with relational databases using Spring Data JPA
- Implement authentication and authorization
- Write comprehensive tests for your applications
- Deploy applications using Docker and cloud platforms
- Design and build microservices architectures

Learning Plans

Choose a plan that fits your schedule. All plans cover the same content.

Intensive

Fast Track

40-50 hrs/week

280-350 total hrs

Full-time immersive learning. Ideal for career changers who can dedicate themselves completely to becoming a backend developer.

Ideal for:

- Career changers with dedicated time
- Recent graduates
- Bootcamp-style learners
- Those preparing for backend interviews

Balanced

Recommended

15-20 hrs/week

300-400 total hrs

Balanced pace allowing deep understanding while maintaining other commitments. The recommended path for most learners.

Ideal for:

- Working professionals
- Students with coursework
- Frontend devs learning backend
- Self-paced learners

Flexible

Self-paced

8-10 hrs/week

320-400 total hrs

Flexible learning for those with limited availability. More time to practice and absorb complex concepts.

Ideal for:

- Full-time employees
- Parents and caregivers
- Those learning multiple skills
- Weekend learners

Phase Schedule by Plan

Phase	Topic	Intensive	Balanced	Flexible
1	Java Fundamentals	Weeks 1-2	Weeks 1-4	Weeks 1-8
2	Development Environment & Tools	Week 3	Weeks 5-6	Weeks 9-12
3	Spring Core & Spring Boot	Weeks 4-5	Weeks 7-10	Weeks 13-20
4	Building REST APIs	Weeks 6-7	Weeks 11-14	Weeks 21-28
5	Data Persistence	Weeks 8-9	Weeks 15-18	Weeks 29-36
6	Security	Week 10	Weeks 19-22	Weeks 37-44
7	Testing & Production	Weeks 11-12	Weeks 23-26	Weeks 45-52

1 Java Fundamentals

Intensive: Weeks 1-2 | Balanced: Weeks 1-4 | Flexible: Weeks 1-8

Java is the foundation. Before touching Spring, you need solid Java skills. This phase covers core language features, object-oriented programming (the paradigm Spring is built on), and essential APIs you'll use daily. Don't rush this — weak Java fundamentals will haunt you later.

Learning Objectives

- Write syntactically correct Java code
- Understand and apply OOP principles (encapsulation, inheritance, polymorphism, abstraction)
- Work with Collections framework effectively
- Use Java 8+ features (Streams, Lambdas, Optional)
- Handle exceptions properly
- Understand generics and when to use them

Topics to Cover

Java Syntax & Basics

The building blocks of every Java program

- Variables & Data Types
- Operators & Expressions
- Control Flow (if/else, switch)
- Loops (for, while, for-each)
- Methods & Parameters
- Arrays

Object-Oriented Programming

WHY: Spring is deeply OOP. Understanding classes, interfaces, and inheritance is non-negotiable. Dependency Injection only makes sense if you understand objects.

- Classes & Objects
- Constructors
- Encapsulation (getters/setters)
- Inheritance & super
- Polymorphism
- Abstract Classes & Interfaces
- Access Modifiers

Collections Framework

WHY: You'll work with collections constantly — API responses, database results, business logic. Knowing which collection to use affects performance and code clarity.

- List (ArrayList, LinkedList)
- Set (HashSet, TreeSet)
- Map (HashMap, TreeMap)
- Queue & Deque
- Iterators
- Collections utility class

Modern Java Features

WHY: Spring Boot leverages modern Java. Lambdas are everywhere in Spring. Streams make data processing elegant. Optional helps handle nulls properly.

- Lambda Expressions
- Functional Interfaces
- Stream API
- Optional
- Method References
- var keyword

Exception Handling

WHY: Robust applications handle errors gracefully. Spring has specific exception handling mechanisms that build on Java's fundamentals.

- try-catch-finally
- Checked vs Unchecked Exceptions
- Custom Exceptions
- try-with-resources
- Exception Chaining

Hands-on Projects

Library Management System (Console)

Build a console application to manage books, members, and borrowing. Practice OOP by modeling real entities as classes with proper relationships.

OOP Design | Collections | Exception Handling | File I/O

Data Processing Pipeline

Read a CSV file of transactions, process them using Streams (filter, map, reduce), and generate summary reports. Focus on functional programming style.

Streams API | Lambdas | File I/O | Collections

Resources

DOCUMENTATION **Java Official Tutorials**

BOOK **Head First Java**

VIDEO **Java Brains (YouTube)**

Milestone

Complete both projects. You should be able to design a system using classes, use appropriate collections, process data with Streams, and handle errors gracefully — all without looking up basic syntax.

2 Development Environment & Tools

Intensive: Week 3 | Balanced: Weeks 5-6 | Flexible: Weeks 9-12

Professional development requires professional tools. This phase sets up your workspace and teaches you the build tools, version control, and IDE features that make you productive. These skills compound over time — invest in them early.

Learning Objectives

- Set up a professional Java development environment
- Understand Maven/Gradle project structure and dependency management
- Use Git effectively for version control
- Navigate and utilize IDE features efficiently
- Understand the Java ecosystem (JDK versions, distributions)

Topics to Cover

IDE Mastery (IntelliJ IDEA)

WHY: Your IDE is your primary tool. Knowing shortcuts, refactoring tools, and debugging features multiplies your productivity by 10x.

- Project Setup & Structure
- Keyboard Shortcuts
- Code Navigation
- Refactoring Tools
- Debugging
- Git Integration

Maven

WHY: Maven manages dependencies and builds. Every Spring Boot project uses it. Understanding pom.xml is essential for adding libraries and configuring builds.

- POM Structure
- Dependencies & Repositories
- Build Lifecycle
- Plugins
- Profiles
- Multi-module Projects

Gradle (Alternative)

WHY: Gradle is Maven's modern alternative, used by many Spring projects. Knowing both makes you versatile.

- build.gradle Structure
- Dependencies
- Tasks
- Groovy vs Kotlin DSL
- Wrapper

Git & GitHub

WHY: Version control is non-negotiable. Collaboration, code review, and deployment all depend on Git proficiency.

- Basic Commands
- Branching Strategies
- Merge vs Rebase
- Pull Requests
- Resolving Conflicts
- .gitignore

Hands-on Projects

Multi-Module Maven Project

Create a Maven project with multiple modules (core, api, utils). Practice dependency management between modules and understand how large projects are structured.

Maven · Project Structure · Dependency Management

Git Workflow Simulation

Practice a real Git workflow: create branches for features, make commits, create PRs, handle merge conflicts, and maintain a clean history.

Git · Branching · Collaboration · Code Review

Resources

DOCUMENTATION **IntelliJ IDEA Guide**

TUTORIAL **Maven in 5 Minutes**

BOOK **Pro Git Book**

Milestone

Set up a Maven project from scratch, manage dependencies, and demonstrate a clean Git workflow with branches and meaningful commits. You should feel comfortable navigating any Java project.

3 Spring Core & Spring Boot

Intensive: Weeks 4-5 | Balanced: Weeks 7-10 | Flexible: Weeks 13-20

Now we enter Spring territory. This phase covers the foundational concepts that everything else builds on: Inversion of Control, Dependency Injection, and how Spring Boot simplifies configuration. Understanding these deeply prevents countless debugging sessions later.

Learning Objectives

- Understand Inversion of Control (IoC) and why it matters
- Master Dependency Injection patterns
- Work with Spring Beans and their lifecycle
- Understand Spring Boot auto-configuration
- Configure applications using properties and profiles
- Structure Spring applications properly

Topics to Cover

Inversion of Control (IoC)

WHY: IoC is THE core concept. Instead of your code controlling dependencies, Spring controls them. This enables loose coupling, testability, and flexibility. Without understanding IoC, Spring is just magic.

- What Problem Does IoC Solve?
- IoC Container
- ApplicationContext
- Bean Definition
- Component Scanning

Dependency Injection (DI)

WHY: DI is how IoC is implemented. It's how your classes get their dependencies without creating them. This is fundamental to every Spring application.

- Constructor Injection (Preferred)
- Setter Injection
- Field Injection (Avoid)
- @Autowired Deep Dive
- @Qualifier & @Primary
- Circular Dependencies

Spring Beans

WHY: Beans are objects managed by Spring. Understanding scopes, lifecycle, and configuration is essential for proper application design.

- @Component, @Service, @Repository
- Bean Scopes (Singleton, Prototype)
- Bean Lifecycle
- @PostConstruct & @PreDestroy
- @Configuration & @Bean
- Conditional Beans

Spring Boot Fundamentals

WHY: Spring Boot removes boilerplate. Understanding what it auto-configures helps you customize when needed and debug when things go wrong.

- Spring Boot Starters
- Auto-configuration
- application.properties/yml
- Profiles
- @Value & @ConfigurationProperties
- Spring Boot DevTools

Hands-on Projects

Dependency Injection Showcase

Build a notification service with multiple implementations (Email, SMS, Push). Use DI to switch implementations via configuration without changing code. Demonstrate all injection types.

IoC · DI · Interfaces · Configuration

Configurable Application

Create an application that behaves differently based on profiles (dev, test, prod). Use @ConfigurationProperties to type-safe bind external configuration.

Spring Boot · Profiles · Configuration · Properties

Resources

DOCUMENTATION **Spring Framework Documentation**

DOCUMENTATION **Spring Boot Reference**

TUTORIAL **Baeldung Spring Tutorials**

Milestone

Explain IoC and DI to someone without using jargon. Build an application where you can swap implementations just by changing configuration. You should understand what Spring Boot auto-configures and why.

4 Building REST APIs

Intensive: Weeks 6-7 | Balanced: Weeks 11-14 | Flexible: Weeks 21-28

This is where you start building real web services. You'll learn to create RESTful endpoints, handle requests and responses, validate input, manage errors, and document your APIs. This is the core skill of a backend developer.

Learning Objectives

- Design RESTful API endpoints following best practices
- Handle HTTP methods, status codes, and headers properly
- Validate request data effectively
- Implement global exception handling
- Document APIs with OpenAPI/Swagger
- Version APIs appropriately

Topics to Cover

REST Fundamentals

WHY: REST is the standard for web APIs. Understanding its principles (resources, HTTP methods, statelessness) leads to intuitive, predictable APIs.

- REST Principles
- Resources & URIs
- HTTP Methods (GET, POST, PUT, DELETE, PATCH)
- Status Codes
- Headers
- Content Negotiation

Spring MVC Controllers

WHY: Controllers are where HTTP meets your Java code. Mastering annotations and patterns here is essential.

- @RestController vs @Controller
- @RequestMapping & Method Annotations
- @PathVariable & @RequestParam
- @RequestBody & @ResponseBody
- ResponseEntity
- @RequestHeader & @CookieValue

Request Validation

WHY: Never trust client input. Validation prevents bugs, security issues, and data corruption. Spring's validation integrates seamlessly.

- Bean Validation (JSR-380)
- @Valid & @Validated
- Built-in Constraints
- Custom Validators
- Validation Groups
- Error Messages

Exception Handling

WHY: Clean error responses make APIs usable. Global exception handling keeps controllers clean and responses consistent.

- @ExceptionHandler
- @ControllerAdvice
- Custom Exception Classes
- Error Response DTOs
- Problem Details (RFC 7807)
- Logging Errors

API Documentation

WHY: Undocumented APIs are unusable APIs. OpenAPI/Swagger generates interactive documentation automatically.

- OpenAPI Specification
- Springdoc-openapi
- Swagger UI
- Annotations for Documentation
- API Versioning Strategies

Hands-on Projects

Task Management API

Build a complete CRUD API for managing tasks/todos. Implement proper REST conventions, validation, error handling, and Swagger documentation.

REST API Design | Validation | Exception Handling | Documentation

E-commerce Product API

Create an API for product catalog with categories, search, filtering, and pagination. Practice complex request handling and response shaping.

Complex Queries | Pagination | Filtering | DTOs

Resources

DOCUMENTATION **Spring MVC Documentation**

ARTICLE **REST API Design Best Practices**

DOCUMENTATION **Springdoc OpenAPI**

Milestone

Build a fully documented REST API with proper validation and error handling. Someone should be able to use your API just by reading the Swagger UI. Your error responses should be helpful, not cryptic.

5 Data Persistence

Intensive: Weeks 8-9 | Balanced: Weeks 15-18 | Flexible: Weeks 29-36

Most applications need a database. This phase covers Spring Data JPA, Hibernate ORM, database design, and query optimization. You'll learn to model data, write efficient queries, and handle transactions properly.

Learning Objectives

- Design database schemas for applications
- Map Java objects to database tables with JPA/Hibernate
- Write queries using Spring Data JPA repositories
- Understand and use transactions correctly
- Optimize database access patterns
- Handle database migrations

Topics to Cover

Relational Database Fundamentals

WHY: Before using an ORM, understand what it abstracts. Knowing SQL and database design helps you use JPA effectively and debug issues.

- Tables, Rows, Columns
- Primary & Foreign Keys
- Normalization
- Indexes
- Basic SQL (SELECT, INSERT, UPDATE, DELETE)
- Joins

JPA & Hibernate

WHY: JPA is the standard for Java persistence. Hibernate is the most popular implementation. Understanding the abstraction helps you avoid common pitfalls.

- Entities & @Entity
- @Id & Generation Strategies
- Field Mappings
- Relationships (@OneToMany, @ManyToOne, @ManyToMany)
- Lazy vs Eager Loading
- Cascading

Spring Data JPA

WHY: Spring Data JPA eliminates boilerplate. Understanding query derivation and custom queries covers 99% of data access needs.

- JpaRepository Interface
- Query Derivation Methods
- @Query (JPQL & Native)
- Pagination & Sorting
- Projections & DTOs
- Specifications for Dynamic Queries

Transactions

WHY: Transactions ensure data consistency. Misunderstanding them leads to bugs that are hard to reproduce and fix.

- ACID Properties
- @Transactional
- Propagation Levels
- Isolation Levels
- Rollback Rules
- Read-only Transactions

Database Migrations

WHY: Production databases evolve. Flyway/Liquibase ensure schema changes are versioned, repeatable, and safe.

- Why Migrations Matter
- Flyway Basics
- Migration Scripts
- Version Control for Schema
- Rollback Strategies

Hands-on Projects

Blog Platform Backend

Build a blog API with Users, Posts, Comments, and Tags. Practice complex relationships, pagination, and query optimization. Include user-post ownership and cascading deletes.

Entity Relationships | JPA Queries | Transactions | Data Modeling

Inventory Management System

Create an inventory system with Products, Categories, Stock movements, and Suppliers. Focus on transaction management for stock updates and reporting queries.

Complex Queries | Transactions | Aggregations | Migrations

Resources

DOCUMENTATION **Spring Data JPA Reference**

DOCUMENTATION **Hibernate User Guide**

BLOG **Vlad Mihalcea's Blog**

Milestone

Build an application with at least 4 related entities. Write both derived queries and custom JPQL. Demonstrate proper transaction handling. Use Flyway for migrations. Explain the N+1 problem and how to avoid it.

6 Security

Intensive: Week 10 | Balanced: Weeks 19-22 | Flexible: Weeks 37-44

Security is not optional. This phase covers authentication, authorization, password handling, and common security patterns. You'll implement JWT-based auth and understand OAuth2 for third-party integration.

Learning Objectives

- Understand authentication vs authorization
- Implement Spring Security in applications
- Secure REST APIs with JWT tokens
- Implement role-based access control
- Understand OAuth2 and OpenID Connect
- Protect against common vulnerabilities

Topics to Cover

Security Fundamentals

WHY: Before implementing security, understand what you're protecting against. Security is about layers, not silver bullets.

- Authentication vs Authorization
- Common Attack Vectors (OWASP Top 10)
- Password Storage (BCrypt)
- HTTPS & TLS
- CORS
- Security Headers

Spring Security Basics

WHY: Spring Security is powerful but complex. Understanding its filter chain and configuration model is essential.

- Security Filter Chain
- SecurityFilterChain Bean
- Authentication Manager
- UserDetailsService
- Password Encoders
- Method Security (@PreAuthorize)

JWT Authentication

WHY: JWTs are the standard for stateless API authentication. Understanding how they work (and their limitations) is crucial for modern backends.

- JWT Structure (Header, Payload, Signature)
- Creating & Validating JWTs
- Access & Refresh Tokens
- Token Storage Considerations
- JWT Filter Implementation
- Token Expiration & Refresh

OAuth2 & OpenID Connect

WHY: OAuth2 enables 'Login with Google/GitHub'. Understanding the flows helps you integrate with identity providers and build your own.

- OAuth2 Flows
- Authorization Code Flow
- Resource Server Configuration
- Social Login Integration
- OpenID Connect Basics

Hands-on Projects

JWT Authentication System

Implement complete JWT auth: registration, login, token refresh, logout (token invalidation). Include role-based access control for different endpoints.

Spring Security | JWT | Password Hashing | Role-based Access

OAuth2 Resource Server

Build an API that accepts tokens from an OAuth2 provider (like Auth0 or Keycloak). Implement proper token validation and scope-based authorization.

OAuth2 | Token Validation | Scopes | Integration

Resources

DOCUMENTATION **Spring Security Reference**

TUTORIAL **JWT.io**

BOOK **OAuth 2.0 Simplified**

Milestone

Implement JWT authentication from scratch (not copy-paste). Explain the token flow, where tokens should be stored, and why refresh tokens exist. Secure different endpoints for different roles.

7 Testing & Production

Intensive: Weeks 11-12 | Balanced: Weeks 23-26 | Flexible: Weeks 45-52

Professional applications are tested and deployable. This phase covers unit testing, integration testing, containerization with Docker, and deployment strategies. These skills separate hobbyists from professionals.

Learning Objectives

- Write effective unit tests with JUnit and Mockito
- Create integration tests for Spring applications
- Containerize applications with Docker
- Understand CI/CD pipelines
- Deploy to cloud platforms
- Monitor and observe applications in production

Topics to Cover

Unit Testing

WHY: Unit tests catch bugs early and enable refactoring. They're your safety net. Understanding mocking is essential for testing in isolation.

- JUnit 5 Basics
- Assertions
- Test Lifecycle
- Mockito Mocking
- Argument Matchers
- Verifying Interactions

Integration Testing

WHY: Integration tests verify components work together. Spring's test support makes testing controllers, repositories, and full slices straightforward.

- @SpringBootTest
- @WebMvcTest for Controllers
- @DataJpaTest for Repositories
- TestContainers for Databases
- MockMvc for API Testing
- Test Configuration

Docker

WHY: Docker ensures 'it works on my machine' actually means something. Containerization is the standard for deployment.

- Docker Basics
- Dockerfile for Spring Boot
- Multi-stage Builds
- Docker Compose
- Environment Variables
- Health Checks

CI/CD & Deployment

WHY: Automated pipelines ensure consistent, reliable deployments. Manual deployment doesn't scale.

- GitHub Actions Basics
- Build & Test Pipelines
- Docker Registry
- Kubernetes Intro
- Cloud Platforms (AWS, GCP, Azure)
- Environment Management

Observability

WHY: You can't fix what you can't see. Logging, metrics, and tracing are essential for production systems.

- Structured Logging (Logback)
- Spring Boot Actuator
- Metrics with Micrometer
- Health Endpoints
- Distributed Tracing Basics

Hands-on Projects

Fully Tested API

Take a previous project and achieve 80%+ test coverage. Write unit tests for services, integration tests for controllers, and use TestContainers for database tests.

JUnit | Mockito | Integration Testing | TestContainers

Dockerized Microservice

Containerize an application with Docker, set up docker-compose with the app and database, create a GitHub Actions pipeline that builds, tests, and pushes the image.

Docker | Docker Compose | CI/CD | GitHub Actions

Resources

DOCUMENTATION **Spring Boot Testing**

DOCUMENTATION **TestContainers**

DOCUMENTATION **Docker Documentation**

Milestone

Have a project with comprehensive tests and a working CI/CD pipeline. Deploy your application somewhere publicly accessible. Demonstrate that you can debug issues using logs and metrics.

Your Path Forward

Career Opportunities

- ✓ Backend Developer
- ✓ Java Developer
- ✓ Full Stack Developer
- ✓ Software Engineer
- ✓ API Developer
- ✓ Microservices Engineer

Tips for Success

- Don't skip Java fundamentals to get to Spring faster — it will backfire
- Understand IoC and DI deeply; everything in Spring builds on these concepts
- Read the official Spring documentation — it's excellent
- Practice debugging; learn to read stack traces and use your debugger
- Write tests from the beginning, not as an afterthought
- Learn SQL independently of JPA; you'll need it for complex queries and debugging
- Join communities: r/java, r/springframework, Spring Discord
- Build projects that solve real problems, not just tutorials
- Contribute to open source Java projects to learn from experts
- Stay updated — Java and Spring release frequently with valuable features

Next Steps

1. Install JDK 17+ and IntelliJ IDEA Community Edition
2. Choose your learning pace (intensive, balanced, or flexible)
3. Start with Phase 1 — resist the urge to jump to Spring
4. Set up a GitHub account and commit code from day one
5. Join the Spring community for help and motivation
6. Build something you actually want to use

Ready to Start Your Journey?

Everything in this roadmap is free to learn on your own. But if you'd like structured guidance, hands-on projects, and mentor support — we'd love to have you at Crio.Do

Explore Crio.Do

https://crio.do